

Object Oriented Programming with C++

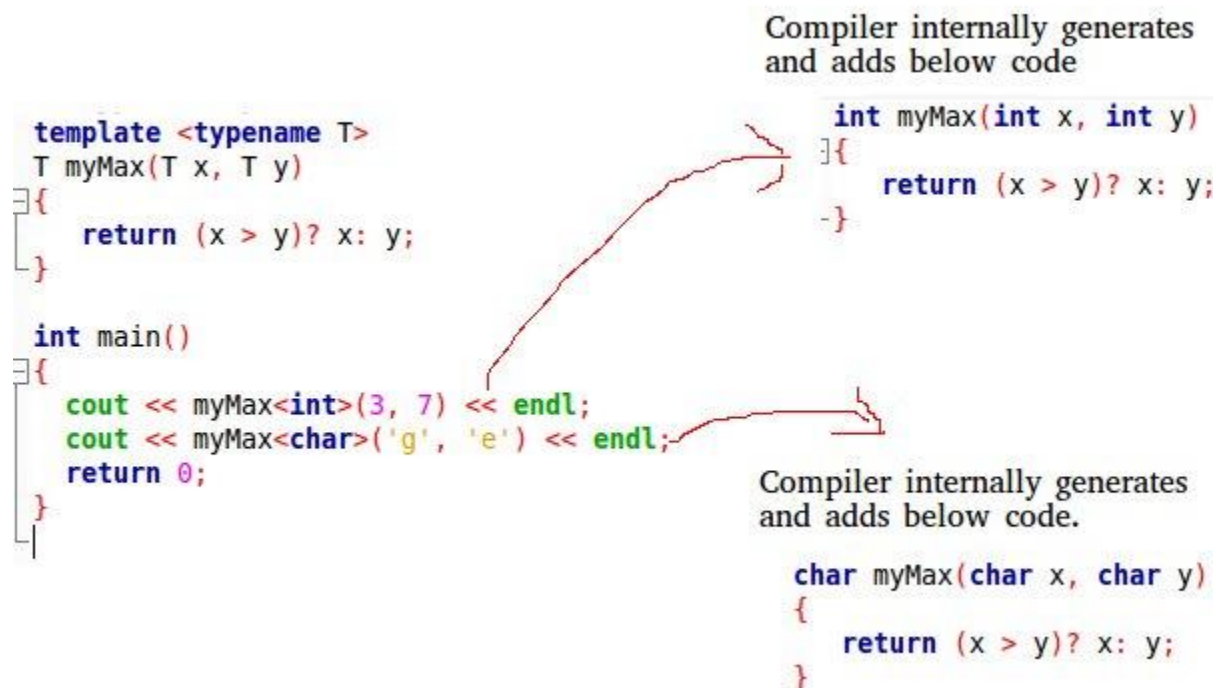
Unit 4

Templates in C++

- A template is a simple and yet very powerful tool in C++.
- The simple idea is to pass data type as a parameter so that we don't need to write the same code for different data types.
- For example, a software company may need sort() for different data types. Rather than writing and maintaining the multiple codes, we can write one sort() and pass data type as a parameter. C++ adds two new keywords to support templates: 'template' and 'typename'.
- The second keyword can always be replaced by keyword 'class'.

How do templates work?

- Templates are expanded at compiler time.
- This is like macros.
- The difference is, the compiler does type checking before template expansion. The idea is simple, source code contains only function/class, but compiled code may contain multiple copies of same function/class.



Function Templates

We write a generic function that can be used for different data types. Examples of function templates are sort(), max(), min(), printArray().

```
#include <iostream>
using namespace std;

// One function works for all data types. This would work
// even for user defined types if operator '>' is overloaded
template<typename T>
T myMax(T x, T y)
{
    return (x > y)? x: y;
}

int main()
{
    cout << myMax<int>(3, 7) << endl; // Call myMax for int
    cout << myMax<double>(3.0, 7.0) << endl; // call myMax for double
    cout << myMax<char>('g', 'e') << endl; // call myMax for char

    return 0;
}
```

Output:

7
7
g

Below is the program to implement [Bubble Sort](#) using templates in C++:

```
// CPP code for bubble sort
// using template function
#include <iostream>
using namespace std;

// A template function to implement bubble sort.
// We can use this for any data type that supports
// comparison operator < and swap works for it.
template<class T>
void bubbleSort(T a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = n - 1; i < j; j--)
            if (a[j] < a[j - 1])
                swap(a[j], a[j - 1]);
}

// Driver Code
int main() {
    int a[5] = {10, 50, 30, 40, 20};
    int n = sizeof(a) / sizeof(a[0]);
```

```

// calls template function
bubbleSort<int>(a, n);

cout << " Sorted array : ";
for (int i = 0; i < n; i++)
    cout << a[i] << " ";
cout << endl;

return 0;
}

```

Output:

Sorted array : 10 20 30 40 50

Class Templates

Like function templates, class templates are useful when a class defines something that is independent of the data type. Can be useful for classes like LinkedList, BinaryTree, Stack, Queue, Array, etc. Following is a simple example of template Array class.

```

#include <iostream>
using namespace std;

template <typename T>
class Array {
private:
    T *ptr;
    int size;
public:
    Array(T arr[], int s);
    void print();
};

template <typename T>
Array<T>::Array(T arr[], int s) {
    ptr = new T[s];
    size = s;
    for (int i = 0; i < size; i++)
        ptr[i] = arr[i];
}

template <typename T>
void Array<T>::print() {
    for (int i = 0; i < size; i++)
        cout << " "<< *(ptr + i);
    cout << endl;
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
}

```

```
Array<int> a(arr, 5);  
a.print();  
return 0;  
}
```

Output:

1 2 3 4 5

Can there be more than one arguments to templates?

Yes, like normal parameters, we can pass more than one data types as arguments to templates. The following example demonstrates the same.

```
#include<iostream>  
using namespace std;  
  
template<class T, class U>  
class A {  
    T x;  
    U y;  
public:  
    A() { cout<<"Constructor Called"<<endl; }  
};  
  
int main() {  
    A<char, char> a;  
    A<int, double> b;  
    return 0;  
}
```

Output:

Constructor Called
Constructor Called

Can we specify default value for template arguments?

Yes, like normal parameters, we can specify default arguments to templates. The following example demonstrates the same.

```
#include<iostream>  
using namespace std;  
  
template<class T, class U = char>  
class A {  
public:  
    T x;  
    U y;  
    A() { cout<<"Constructor Called"<<endl; }  
};
```

```
int main() {  
    A<char> a; // This will call A<char, char>  
    return 0;  
}
```

Output:

Constructor Called

C++ Files and Streams

In C++ programming we are using the **iostream** standard library, it provides **cin** and **cout** methods for reading from input and writing to output respectively.

To read and write from a file we are using the standard C++ library called **fstream**. Let us see the data types define in **fstream** library is:

Data Type	Description
fstream	It is used to create files, write information to files, and read information from files.
ifstream	It is used to read information from files.
ofstream	It is used to create files and write information to the files.

C++ FileStream example: writing to a file

Let's see the simple example of writing to a text file **testout.txt** using C++ FileStream programming.

```
1. #include <iostream>  
2. #include <fstream>  
3. using namespace std;  
4. int main () {  
5.     ofstream filestream("testout.txt");  
6.     if (filestream.is_open())  
7.     {  
8.         filestream << "Welcome to javaTpoint.\n";  
9.         filestream << "C++ Tutorial.\n";  
10.    filestream.close();  
11.    }  
12.    else cout << "File opening is fail."  
13.    return 0;  
14. }
```

Output:

The content of a text file **testout.txt** is set with the data:
Welcome to javaTpoint.
C++ Tutorial.

C++ FileStream example: reading from a file

Let's see the simple example of reading from a text file **testout.txt** using C++ FileStream programming.

```
1. #include <iostream>
2. #include <fstream>
3. using namespace std;
4. int main () {
5.     string srg;
6.     ifstream filestream("testout.txt");
7.     if (filestream.is_open())
8.     {
9.         while ( getline (filestream,srg) )
10.        {
11.            cout << srg <<endl;
12.        }
13.        filestream.close();
14.    }
15.    else {
16.        cout << "File opening is fail."<<endl;
17.    }
18.    return 0;
19. }
```

Note: Before running the code a text file named as "testout.txt" is need to be created and the content of a text file is given below:

Output:

Welcome to javaTpoint.
C++ Tutorial.

C++ Read and Write Example

Let's see the simple example of writing the data to a text file **testout.txt** and then reading the data from the file using C++ FileStream programming.

```
1. #include <fstream>
2. #include <iostream>
```

```

3. using namespace std;
4. int main () {
5.     char input[75];
6.     ofstream os;
7.     os.open("testout.txt");
8.     cout << "Writing to a text file:" << endl;
9.     cout << "Please Enter your name: ";
10.    cin.getline(input, 100);
11.    os << input << endl;
12.    cout << "Please Enter your age: ";
13.    cin >> input;
14.    cin.ignore();
15.    os << input << endl;
16.    os.close();
17.    ifstream is;
18.    string line;
19.    is.open("testout.txt");
20.    cout << "Reading from a text file:" << endl;
21.    while (getline (is,line))
22.    {
23.        cout << line << endl;
24.    }
25.    is.close();
26.    return 0;
27. }

```

Output:

```

Writing to a text file:
Please Enter your name: Nakul Jain
Please Enter your age: 22
Reading from a text file:  Nakul Jain
22

```

C++ File Pointers and Random Access

Every file maintains two pointers called `get_pointer` (in input mode file) and `put_pointer` (in output mode file) which tells the current position in the file where reading or writing will take place. (A file pointer in this context is not like a C++ pointer but it works like a book-mark in a book.). These pointers help attain random access in file. That means moving directly to any location in the file instead of moving through it sequentially.

There may be situations where random access is the best choice. For example, if you have to modify a value in record no 21, then using random access techniques, you can place the file pointer at the beginning of record 21 and then straight-way process the record. If sequential access is used, then you'll have to unnecessarily go through first twenty records in order to reach at record 21.

The seekg(), seekp(), tellg() and tellp() Functions

- In C++, random access is achieved by manipulating seekg(), seekp(), tellg() and tellp() functions.
- The seekg() and tellg() functions allow you to set and examine the get_pointer, and the seekp() and tellp() functions perform these operations on the put_pointer.
- The seekg() and tellg() functions are for input streams (ifstream) and seekp() and tellp() functions are for output streams (ofstream).
- However, if you use them with an fstream object then tellg() and tellp() return the same value. Also seekg() and seekp() work the same way in an fstream object. The most common forms of these functions are :

seekg()	istream & seekg(long); istream & seekg(long, seek_dir);	Form 1 Form 2
seekp()	ofstream & seekp(long); ofstream & seekp(long, seek_dir);	Form 1 Form 2
tellg()	long tellg()	
tellp()	long tellp()	

The working of seekg() & seekp() and tellg() & tellp() is just the same except that seekg() and tellg() work for ifstream objects and seekp() and tellp() work for ofstream objects. In the above table, seek_dir takes the definition enum seek_dir { beg, cur, end};.

The seekg() or seekp(), when used according to Form 1, then it moves the get_pointer or put_pointer to an absolute position. Here is an example:

```
ifstream fin;  
ofstream fout;  
:  
    // file opening routine  
fin.seekg(30); // will move the get_pointer (in ifstream) to byte number 30 in the file  
fout.seekp(30); // will move the put_pointer (in ofstream) to byte number 30 in the file
```

When seekg() or seekp() function is used according to Form 2, then it moves the get_pointer or put_pointer to a position relative to the current position, following the definition of seek_dir. Since, seek_dir is an enumeration defined in the header file iostream.h, that has the following values:

```
ios::beg,    // refers to the beginning of the file  
ios::cur,    // refers to the current position in the file  
ios::end}    // refers to the end of the file
```

Here is an example.

```
fin.seekg(30, ios::beg);    // go to byte no. 30 from beginning of file linked with fin  
fin.seekg(-2, ios::cur);    // back up 2 bytes from the current position of get pointer  
fin.seekg(0, ios::end);    // go to the end of the file  
fin.seekg(-4, ios::end);    // backup 4 bytes from the end of the file
```


The functions `tellg()` and `tellp()` return the position, in terms of byte number, of `put_pointer` and `get_pointer` respectively, in an output file and input file.

Namespaces in C++

- Consider a situation, when we have two persons with the same name, Zara, in the same class. Whenever we need to differentiate them definitely we would have to use some additional information along with their name, like either the area, if they live in different area or their mother's or father's name, etc.
- Same situation can arise in your C++ applications. For example, you might be writing some code that has a function called `xyz()` and there is another library available which is also having same function `xyz()`. Now the compiler has no way of knowing which version of `xyz()` function you are referring to within your code.
- A **namespace** is designed to overcome this difficulty and is used as additional information to differentiate similar functions, classes, variables etc. with the same name available in different libraries.
- Using namespace, you can define the context in which names are defined. In essence, a namespace defines a scope.

Defining a Namespace

A namespace definition begins with the keyword **namespace** followed by the namespace name as follows –

```
namespace namespace_name {  
    // code declarations  
}
```

To call the namespace-enabled version of either function or variable, prepend (`::`) the namespace name as follows –

```
name::code; // code could be variable or function.
```

Let us see how namespace scope the entities including variable and functions –

versatile

```
#include <iostream>  
using namespace std;  
  
// first name space  
namespace first_space {  
    void func() {  
        cout << "Inside first_space" << endl;  
    }  
}  
  
// second name space  
namespace second_space {  
    void func() {
```

```

        cout << "Inside second_space" << endl;
    }
}

int main () {
    // Calls function from first name space.
    first_space::func();

    // Calls function from second name space.
    second_space::func();

    return 0;
}

```

If we compile and run above code, this would produce the following result –

```

Inside first_space
Inside second_space

```

The using directive

You can also avoid prepending of namespaces with the **using namespace** directive. This directive tells the compiler that the subsequent code is making use of names in the specified namespace. The namespace is thus implied for the following code –

```

#include <iostream>
using namespace std;

// first name space
namespace first_space {
    void func() {
        cout << "Inside first_space" << endl;
    }
}

// second name space
namespace second_space {
    void func() {
        cout << "Inside second_space" << endl;
    }
}

using namespace first_space;
int main () {
    // This calls function from first name space.
    func();

    return 0;
}

```

If we compile and run above code, this would produce the following result –

```

Inside first_space

```

C++ Exception Handling

C++ Exceptions

When executing C++ code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.

When an error occurs, C++ will normally stop and generate an error message. The technical term for this is: C++ will throw an **exception** (throw an error).

C++ try and catch

Exception handling in C++ consist of three keywords: `try`, `throw` and `catch`:

The `try` statement allows you to define a block of code to be tested for errors while it is being executed.

The `throw` keyword throws an exception when a problem is detected, which lets us create a custom error.

The `catch` statement allows you to define a block of code to be executed, if an error occurs in the try block.

The `try` and `catch` keywords come in pairs:

Example

```
try {  
    // Block of code to try  
    throw exception; // Throw an exception when a problem arise  
}  
catch () {  
    // Block of code to handle errors  
}
```

Consider the following example:

Example

```
try {  
    int age = 15;  
    if (age >= 18) {  
        cout << "Access granted - you are old enough."  
    } else {  
        throw (age);  
    }  
}
```

```
}  
catch (int myNum) {  
    cout << "Access denied - You must be at least 18 years old.\n";  
    cout << "Age is: " << myNum;  
}
```

Example explained

We use the `try` block to test some code: If the `age` variable is less than 18, we will `throw` an exception, and handle it in our `catch` block.

In the `catch` block, we catch the error and do something about it. The `catch` statement takes a **parameter**: in our example we use an `int` variable (`myNum`) (because we are throwing an exception of `int` type in the `try` block (`age`)), to output the value of `age`.

If no error occurs (e.g. if `age` is 20 instead of 15, meaning it will be greater than 18), the `catch` block is skipped:

Example

```
int age = 20;
```

You can also use the `throw` keyword to output a reference number, like a custom error number/code for organizing purposes:

Example

```
try {  
    int age = 15;  
    if (age >= 18) {  
        cout << "Access granted - you are old enough.";  
    } else {  
        throw 505;  
    }  
}  
catch (int myNum) {  
    cout << "Access denied - You must be at least 18 years old.\n";  
    cout << "Error number: " << myNum;  
}
```

Handle Any Type of Exceptions (...)

If you do not know the `throw` **type** used in the `try` block, you can use the "three dots" syntax (`...`) inside the `catch` block, which will handle any type of exception:

Example

```
try {
    int age = 15;
    if (age >= 18) {
        cout << "Access granted - you are old enough.";
    } else {
        throw 505;
    }
}
catch (...) {
    cout << "Access denied - You must be at least 18 years old.\n";
}
```

Uncaught exception In C++

Usually, the exception is handled by the try-catch block, but there are instances where there isn't a matching catch block and the program just terminates. This `terminate()` function is modifiable as per user requirements.

Whenever an exception arises in C++, it is handled as per the behavior defined using the try-catch block.

However, there is often the case when an exception is thrown but isn't caught because the exception handling subsystem fails to find a matching catch block for that particular exception. In that case, the following set of actions takes place:

1. The exception handling subsystem calls the function: **unexpected()**. This function, provided by the default C++ library, defines the behavior when an uncaught exception arises. By default, `unexpected` calls **terminate()**.
2. The `terminate` function defines the actions that are to be performed during process termination. This, by default, calls **abort()**.
3. The process is aborted.

The `terminate()` and `unexpected()` simply call other functions to actually handle an error. As explained above, `terminate` calls `abort()`, and `unexpected()` calls `terminate()`.

Example

```
#include <exception>
#include <iostream>
using namespace std;
//defining custom terminator
void myhandler() {
    cout << "Inside new terminate handler\n";
    abort();
}
int main() {
    set_terminate(myhandler);
}
```

```
try {  
    cout << "Inside try block\n";  
    throw 100;  
}  
catch (char a){  
    cout << "Inside catch block\n";  
}  
return 0;  
}
```

Output

Inside try block
Inside new terminate handler